

$LBSC^+$: Learning-based Cost-aware Caching with Performance Optimization for Cloud Databases

Zhongle Xie*, Zhaoxuan Ji*, Kehan Liu, Peng Lu *Member, IEEE*, Quanqing Xu, Meihui Zhang *Senior Member, IEEE*

Abstract—Cloud databases rely on caching to mitigate high latency and limited bandwidth introduced by storage-disaggregation architectures. However, existing caching mechanisms struggle to handle dynamic workloads, ignore data-fetching costs, and optimize only eviction, leading to inefficient cache utilization. Learning-based approaches offer adaptability but incur high inference overhead and typically assume unconditional item admission, resulting in unnecessary evictions and wasted computation. In this paper, we propose $LBSC^+$, a learning-based cost-aware caching framework that jointly optimizes admission and eviction for cloud databases. First, we introduce Belady-SizeCost, an approximately optimal cost-aware oracle for joint admission and eviction, which prioritizes data items with high cost per byte that are likely to be accessed in the near future. Using this oracle, we then train a lightweight supervised model to predict the next-reuse distance of cached items, which is then combined with item cost and size to estimate a unified utility score for admission and eviction. To reduce inference overhead during online execution, we further incorporate three performance optimizations for $LBSC^+$: adaptive inference reuse, cost-aware sampling, and model reuse. Experiments using synthetic traces and real-world cloud database deployments show that $LBSC^+$ reduces total data-fetching cost by up to 17% relative to state-of-the-art methods. Additionally, the proposed performance optimization techniques reduce computation overhead by up to 90%, making $LBSC^+$ practical for real-world cloud systems.

Index Terms—cloud database, caching, machine learning

I. INTRODUCTION

CLOUD databases [1]–[3] increasingly adopt storage-disaggregation architectures, where compute and storage nodes communicate over the network. While this architecture provides elasticity and operational flexibility, it also introduces high data-access latency and limited bandwidth between compute and storage. To mitigate these bottlenecks, cloud systems often cache data at compute nodes. However, traditional caching techniques are ill-suited for cloud environments because they fail to satisfy three essential requirements:

Zhongle Xie is with Zhejiang University, Hangzhou 310000, China (e-mail: xiezl@zju.edu.cn)

Zhaoxuan Ji is with Beijing Institute of Technology, Beijing 100081, China (e-mail: jizhaoxuan@bit.edu.cn)

Kehan Liu is with Zhejiang University, Ningbo 315000, China (e-mail: 22551107@zju.edu.cn)

Peng Lu is with Zhejiang University, Ningbo 315000, China (e-mail: peng.lu@zju.edu.cn)

Quanqing Xu is with OceanBase and the Research Institute of Ant Group, Hangzhou 310000, China (e-mail: xuquanqing.xqq@oceanbase.com)

Meihui Zhang is with Beijing Institute of Technology, Beijing 100081, China (e-mail: meihui_zhang@bit.edu.cn)

Corresponding author: Meihui Zhang

* The authors contributed equally

The first requirement is **Adaptability to dynamic workloads**. Cloud databases experience highly dynamic workloads compared to conventional databases [4]. Real-world traces often exhibit mixed patterns, such as random lookups, large scans, and skewed access bursts that shift over time [5]. Heuristic policies like Least Recently Used (LRU) [6] perform well only on specific access patterns and degrade significantly when the workload changes [5]. Consequently, caching strategies relying on those static heuristics cannot consistently adapt to evolving request behaviors.

The second is **Awareness of data-fetching cost**. The data fetching cost in cloud databases comprises both the transfer costs, dominated by bandwidth limitations [7], [8], and computation costs, introduced by predicate push-down or object processing in storage nodes [9], [10]. These costs may vary by data items and queries. Without explicit cost awareness, cache management decisions may retain low-utility items and evict expensive ones, causing repeated high-cost data fetching, resulting in suboptimal performance.

The last requirement is **Coordinated admission and eviction**. Caches for cloud databases must jointly determine *what to admit* and *what to evict*. Existing approaches, including LBSC [11], focus primarily on eviction and implicitly admit every incoming item. This strategy may lead to cache pollution: a burst of low-utility or one-time requests evicts valuable items unnecessarily. Since admission and eviction decisions directly affect one another, cloud systems require a unified mechanism that simultaneously optimizes both.

Recently, numerous learning-based algorithms have been developed to enhance the adaptability of caching in traditional databases [5], [12], [13], typically by mimicking the Belady [14] oracle algorithm. Yet, most of these methods still lack cost-awareness and consider only evictions. In addition, learning-based systems introduce non-negligible inference and retraining overhead, which may even outweigh their benefits in latency-sensitive settings. These observations motivate the need for a cost-aware, learning-based caching framework that jointly optimizes admission and eviction while remaining efficient for production cloud databases. Designing such a framework, however, faces two challenges:

Challenge 1: The absence of an oracle algorithm for joint admission and eviction. As in prior learning-based caching algorithms [13], [15], [16], incorporating an oracle algorithm is essential for cloud-native databases. However, deriving an optimal oracle becomes challenging when considering data fetching costs with joint admission and eviction. In fact, designing the optimal caching policy in such a case is known

to be an NP-hard problem [17].

Challenge 2: The efficiency in utilizing learning models.

Although existing learning-based algorithms adapt well to changing workloads, they typically require continuous re-training and extensive model inferences [13], [16], which introduces substantial computational overhead. Without careful optimization, such overheads may outweigh the benefits of improved cache decisions, limiting applicability in latency-sensitive systems.

To address these challenges, we propose $LBSC^+$, a unified learning-based cost-aware caching framework for cloud databases. $LBSC^+$ jointly predicts admission and eviction decisions using a lightweight supervised model trained on a new oracle algorithm we introduce, called BeladySizeCost, which approximates the optimal joint policy while incorporating data-fetching cost. The oracle prioritizes items with high cost per byte and high likelihood of near-future access, enabling principled cost-aware caching. To make $LBSC^+$ practical in real systems, we further introduce three complementary performance optimizations, i.e., adaptive inference reuse, cost-aware sampling, and model reuse, which together reduce online computation overhead by up to 90%. Through extensive experiments on synthetic traces and real cloud databases, we show that $LBSC^+$ reduces total data-fetching cost by up to 17% and delivers robust performance under diverse workloads.

To summarize, we make the following contributions in this work:

- We present $LBSC^+$, a unified learning-based framework, which jointly optimizes admission and eviction for cost-aware caching against dynamic workloads in cloud databases.
- We propose BeladySizeCost, a cost-aware oracle algorithm that guides both admission and eviction decisions in our learning-based framework. **We establish that, BeladySizeCost achieves a $(1 - \epsilon)$ -approximation when individual items are small relative to the cache capacity.**
- We design a lightweight shared model for both admission and eviction predictions and introduce inference reuse, sampling, and model reuse to reduce computation overhead without sacrificing accuracy of the model.
- We implement an $LBSC^+$ simulator and integrate $LBSC^+$ into underlying cloud databases. Experiments on real cloud workloads demonstrate that $LBSC^+$ improves performance by up to 17% compared with state-of-the-art algorithms, and reduces computation overhead by up to 90%.

The rest of the paper is organized as follows. Section II explains why we need a new metric for caching algorithms and introduces the oracle BeladySizeCost algorithm. Section III and IV detail the $LBSC^+$ framework. System integration and extension is discussed in Section V. Section VI evaluates the performance of our framework compared to the baselines. We review the related work in Section VII and conclude the paper in Section VIII.

II. BELADYSIZECOST: COST-AWARE ORACLE

TABLE I
PERFORMANCE METRICS USED IN CACHING POLICIES

Metrics	Definition
Miss Rate	$\frac{\sum_{i \in Hit} 0 + \sum_{j \in Miss} 1}{\sum_{i \in Hit} 1 + \sum_{j \in Miss} 1}$
Byte Miss Rate	$\frac{\sum_{i \in Hit} 0 + \sum_{j \in Miss} s_j}{\sum_{i \in Hit} s_i + \sum_{j \in Miss} s_j}$
Total Cost	$\sum_{j \in Miss} C_j$

ALGORITHM IN CACHING

In this section, we first motivate the need for a cost-aware metric in cloud database caching and then introduce BeladySizeCost, an approximately optimal oracle algorithm designed to minimize total data-fetching cost. We formally describe its ranking function, theoretical basis, and practical estimation strategy.

A. Cost-aware Metric in Caching

In cloud databases, the data-fetching costs consist of transfer cost, dominated by bandwidth constraints and large data volume, and computation cost, incurred by operations on storage nodes. Traditional caching systems evaluate performance using metrics such as miss rate or byte miss rate as summarized in Table I. The former, used in databases like MySQL [18] and PostgreSQL [19], counts the fraction of cache misses but neglects the data-fetching costs. The latter improves upon this by accounting for cache item sizes, thereby reflecting transfer costs. However, it still assumes a uniform retrieval cost across cached items and overlooks computation costs. The assumptions do not always hold for cutting-edge cloud databases, especially those built on storage-disaggregated architectures. To more accurately represent data-fetching cost in such environments, we define the *total cost* of a cached item i , denoted as C_i , as follows:

$$C_i = CC_i + TC_i \quad (1)$$

In the equation, CC_i presents the computation cost, which signifies the time required for calculating the cached data (e.g., intermediate results of the query). TC_i denotes the transfer cost, which reflects the time to transfer data between two nodes, and it is proportional to the data size.

B. Cost-aware Caching Oracle

1) *Global Optimization Formulation*: The Belady algorithm [14] is recognized as the optimal offline eviction policy when cache items have uniform sizes, identical fetching costs and unconditional admission. It evicts the item whose next access lies farthest in the future and has been widely used as the oracle for learning-based caching algorithms [13], [20].

However, Belady becomes inadequate for cloud databases because these assumptions no longer hold. In storage-disaggregated architectures, cache items vary in size and incur heterogeneous *total cost* defined in Eq. 1. Moreover, unconditional admission may introduce cache pollution as mentioned in Section I. Motivated by this, we summarize the

desired properties of a practical oracle for cloud databases as follows:

- P1. **Cost-awareness.** The oracle should explicitly consider **variant** data-fetching costs and item sizes with admission control.
- P2. **Efficient cache decision.** Since the oracle is used to continuously generate supervision for learning-based caching algorithms, its decisions should be efficiently derived without requiring an expensive global optimization for all requests.
- P3. **Belady reduction.** When all cached items have identical fetching costs and sizes, the oracle should revert to the Belady policy.

To meet these properties, we propose BeladySizeCost, an approximately optimal cost-aware oracle algorithm for cloud databases. The key idea is to quantify the benefit of caching each item based on its cost, its **likelihood** of near-future access, and its size. Consider a workload

$$\mathcal{R} = (\alpha_1, \dots, \alpha_t, \dots, \alpha_T) \quad (2)$$

where t denotes the request index and α_t denotes the item requested at index t . At request α_t , for each cached item i , BeladySizeCost computes a utility score:

$$U_i^t = \frac{C_i \cdot p_i^t}{s_i} \quad (3)$$

where C_i denotes the *total cost* of item i defined in Eq. 1, p_i^t represents the **likelihood** that data item i will be reused in the future (future-reuse likelihood) at request α_t , and s_i is the data size. Upon each request arrival, BeladySizeCost performs joint admission and eviction through a unified ranking process. Specifically, it forms a **candidate set** consisting of all cached items and the newly requested item, computes their utility scores, and removes the lowest-utility items until the cache capacity constraint is satisfied.

Next, we describe the caching objective and how it maps to BeladySizeCost. Let S_t denote the set of cached items immediately before processing the request at index t , and let CS denote the cache capacity. The request at index t incurs the data-fetching cost C_{α_t} if $\alpha_t \notin S_t$, and incurs no data-fetching cost otherwise. Therefore, the objective is to minimize the accumulated **total cost** of the workload, i.e.,

$$\min \sum_{t=1}^T C_{\alpha_t} \cdot \mathbb{I}(\alpha_t \notin S_t) \quad (4)$$

where $\mathbb{I}(\cdot)$ is the indicator function, i.e., $\mathbb{I}(true) = 1$ while $\mathbb{I}(false) = 0$. In other words, every cache miss contributes the *total cost* of the workload. The caching algorithm must determine whether item α_t should be admitted and, if necessary, which existing items should be evicted. The cache decision arises when $\sum_{i \in S_t \cup \{\alpha_t\}} s_i > CS$.

In this case, the candidate set is $\mathcal{T}_t(S_t) = S_t \cup \{\alpha_t\}$. Eq. 4 gives the global objective over the entire request sequence. However, computing the exact contribution of each candidate to Eq. 4 would require jointly considering all subsequent accesses and cache-state transitions, which leads to the globally coupled optimization discussed above. We therefore construct

a **per-request surrogate** that captures the immediate cost-saving potential of each candidate item. Specifically, retaining item i can avoid its fetching cost C_i only if it is reused in the future. Moreover, an earlier reuse represents a more immediate opportunity for such a saving. We define $C_i \cdot p_i^t$ as the **cost-saving benefit** of retaining item i at request α_t . The resulting per-request decision is:

$$\begin{aligned} \max \quad & \sum_{i \in \mathcal{T}_t(S_t)} C_i \cdot p_i^t \cdot x_i \\ \text{s.t.} \quad & \sum_{i \in \mathcal{T}_t(S_t)} s_i \cdot x_i \leq CS \end{aligned} \quad (5)$$

where $x_i = 1$ indicates that item i is retained in the cache, while $x_i = 0$ indicates that it is evicted. Eq. 5 is a 0-1 knapsack problem and is therefore NP-hard [21]. Eq. 4 further couples these decisions across the request sequence through the evolving cache state, making an exact global solution unsuitable for continuously generating supervision [17]. Thus, directly solving this global optimization problem is impractical as a supervision signal and therefore fails to satisfy P2.

2) *BeladySizeCost Algorithm:* Next, we show that BeladySizeCost provides an approximation guarantee for the per-request optimization problem in Eq. 5.

Theoretical Analysis. The intuition behind BeladySizeCost is that cached items in cloud databases are typically much smaller than the entire cache capacity [7], [8]. Therefore, we have the following assumption:

Assumption 1. For a small constant $0 < \epsilon < 1$, the cache satisfies $s_{max} \leq \epsilon \cdot CS$, where s_{max} is the maximum size of any item.

Let S_t^{BSC} denote the cached items selected by BeladySizeCost after processing the request at index t . We have the following lemma:

Lemma 1. Under Assumption 1, if $\{S_t \cup \{\alpha_t\}\}$ exceeds the cache capacity and BeladySizeCost repeatedly removes the item with the lowest utility until the capacity constraint is satisfied, the resulting cache S_t^{BSC} satisfies $\sum_{i \in S_t^{BSC}} s_i > (1 - \epsilon) \cdot CS$.

Proof. Let j denote the last item removed by BeladySizeCost. The cache capacity is exceeded before j is removed, i.e., $\sum_{i \in S_t^{BSC}} s_i + s_j > CS$, where s_j is the size of item j . Rearranging this inequality gives $\sum_{i \in S_t^{BSC}} s_i > CS - s_j$. According to Assumption 1, $s_j \leq \epsilon \cdot CS$. Hence, $\sum_{i \in S_t^{BSC}} s_i > (1 - \epsilon) \cdot CS$. $\square$

Lemma 1 shows that the removal of BeladySizeCost leaves the cache close to fully utilized when individual items are small relative to the cache. We next characterize the approximate optimality of BeladySizeCost selection.

Theorem 1. Let S_t^* be cached items selected by the solution to Eq. 5 after processing the request at index t . Under Assumption 1, BeladySizeCost achieves a $(1 - \epsilon)$ -approximation to the optimal solution of Eq. 5, i.e.,

$$\sum_{i \in S_t^{BSC}} C_i \cdot p_i^t > (1 - \epsilon) \sum_{i \in S_t^*} C_i \cdot p_i^t \quad (6)$$

Proof. Let j be the last item removed by BeladySizeCost. Since BeladySizeCost removes items in nondecreasing order of $U_i^t = \frac{C_i \cdot p_i^t}{s_i}$, all items retained in S_t^{BSC} have utility scores no smaller than U_j^t . Thus,

$$\sum_{i \in S_t^{BSC}} C_i \cdot p_i^t = \sum_{i \in S_t^{BSC}} U_i^t \cdot s_i \geq U_j^t \sum_{i \in S_t^{BSC}} s_i \quad (7)$$

which gives

$$U_j^t \leq \frac{\sum_{i \in S_t^{BSC}} C_i \cdot p_i^t}{\sum_{i \in S_t^{BSC}} s_i} \quad (8)$$

Note that Eq. 5 is a standard 0-1 knapsack problem. To bound its optimum, we consider the relaxation of Eq. 5 by replacing $x_i \in \{0, 1\}$ with $0 \leq x_i \leq 1$, i.e., the fractional knapsack problem [22]. Let OPT_F denote the optimal solution of fractional knapsack, and it selects items in nonincreasing order of value density (i.e., utility $\frac{C_i \cdot p_i^t}{s_i}$) [22]. Since S_t^{BSC} contains the highest-utility items and j is the next item in this order, OPT_F consists of all items in S_t^{BSC} , followed by a fraction of j that fills the remaining capacity. Therefore,

$$OPT_F = \sum_{i \in S_t^{BSC}} C_i \cdot p_i^t + U_j^t \cdot (CS - \sum_{i \in S_t^{BSC}} s_i) \quad (9)$$

Substituting Eq. 8 into Eq. 9 gives

$$\begin{aligned} OPT_F &\leq \sum_{i \in S_t^{BSC}} C_i \cdot p_i^t + \frac{\sum_{i \in S_t^{BSC}} C_i \cdot p_i^t}{\sum_{i \in S_t^{BSC}} s_i} (CS - \sum_{i \in S_t^{BSC}} s_i) \\ &= \left(\sum_{i \in S_t^{BSC}} C_i \cdot p_i^t \right) \frac{CS}{\sum_{i \in S_t^{BSC}} s_i} \end{aligned} \quad (10)$$

According to Lemma 1, we have

$$\frac{CS}{\sum_{i \in S_t^{BSC}} s_i} < \frac{CS}{(1 - \epsilon) \cdot CS} = \frac{1}{1 - \epsilon} \quad (11)$$

Combining Eqs. 10 and 11 yields

$$\sum_{i \in S_t^{BSC}} C_i \cdot p_i^t > (1 - \epsilon) \cdot OPT_F \quad (12)$$

Note that the fractional knapsack is a relaxation of the original 0-1 problem: every feasible binary solution $x_i \in \{0, 1\}$ is also feasible when $0 \leq x_i \leq 1$. Therefore, enlarging the feasible set cannot decrease the optimal value [22], and thus

$$\sum_{i \in S_t^{BSC}} C_i \cdot p_i^t > (1 - \epsilon) \cdot OPT_F \geq (1 - \epsilon) \sum_{i \in S_t^*} C_i \cdot p_i^t. \quad (13)$$

□

Future-reuse likelihood Estimation. The utility score in Eq. 3 contains three factors. Among them, the item size s_i and data-fetching cost C_i can be directly obtained from the system, while the future-reuse likelihood p_i^t needs to be estimated. We derive p_i^t from **temporal locality**. Intuitively, an item that will be reused soon should receive a higher caching priority than an item whose next reuse lies far in the future, since retaining the former is more likely to avoid an upcoming data-fetching cost. Ideally, the future reuse tendency of item i could be characterized by all of its future accesses. Let α_{t+x} denote

the item requested at index $t + x$, i.e., x requests after request index t . We can conceptually define the reuse likelihood as

$$p_i^t \propto \sum_{x=1}^{\infty} \frac{1}{x} \cdot \mathbb{I}(\alpha_{t+x} = i) \quad (14)$$

However, explicitly predicting all future accesses requires estimating both how many times an item will be reused and when these accesses will occur. Such long-horizon prediction substantially increases model complexity and prediction uncertainty, making it unsuitable as a lightweight supervision signal. We therefore approximate the above reuse likelihood using only the **next-reuse distance**, which is the future access most directly affected by the current caching decision. For an item i at request index t , let $d_i^t = \min\{x > 0 \mid \alpha_{t+x} = i\}$ denote the number of requests until item i is accessed again. If i does not appear again in the remaining trace, we set $d_i^t = \infty$. A smaller d_i^t indicates stronger temporal locality and a more immediate opportunity to avoid a future fetching cost. We therefore use the reciprocal of the next reuse distance as a simple, parameter-free discounted reuse score:

$$p_i^t = \begin{cases} \frac{1}{d_i^t}, & d_i^t < \infty, \\ 0, & d_i^t = \infty \end{cases} \quad (15)$$

BeladySizeCost reevaluates the cache state at every request. Each time item i is accessed, its next-reuse information is updated according to the next occurrence of i in the remaining workload. Thus, later accesses are not permanently discarded. They are progressively incorporated through subsequent cache decisions.

To summarize, we discuss how BeladySizeCost satisfies the proposed properties, i.e., P1-P3. For P1, it explicitly incorporates *total cost* in the utility score. For P2, it relies only on per-item statistics, computing with low overhead. For P3, if all items share identical cost and size, the factor $\frac{C_i}{s_i}$ is constant across items. The utility score $U_i^t = \frac{C_i}{s_i} \cdot \frac{1}{d_i^t}$ is therefore equivalent to ranking by $\frac{1}{d_i^t}$ for items with future access and 0 otherwise. Hence, items with larger next-access distance receive smaller utility and are evicted first, which matches the Belady algorithm.

III. DESIGN OF $LBSC^+$

We present the design of $LBSC^+$, including an overview of the framework, the workflow of the Cache Manager, and the learning-based Predictor responsible for generating admission and eviction decisions, in this section.

A. Overview

$LBSC^+$, as depicted in Figure 1, consists of three tightly integrated components: Cache Manager, Predictor, and Performance Optimizer. The framework receives a continuous stream of requests, and for each request, $LBSC^+$ determines whether the item should be admitted into the cache and, if necessary, which existing items should be evicted when the cache becomes full. The design is centered on a single learning-based model trained to approximate the optimal behavior of

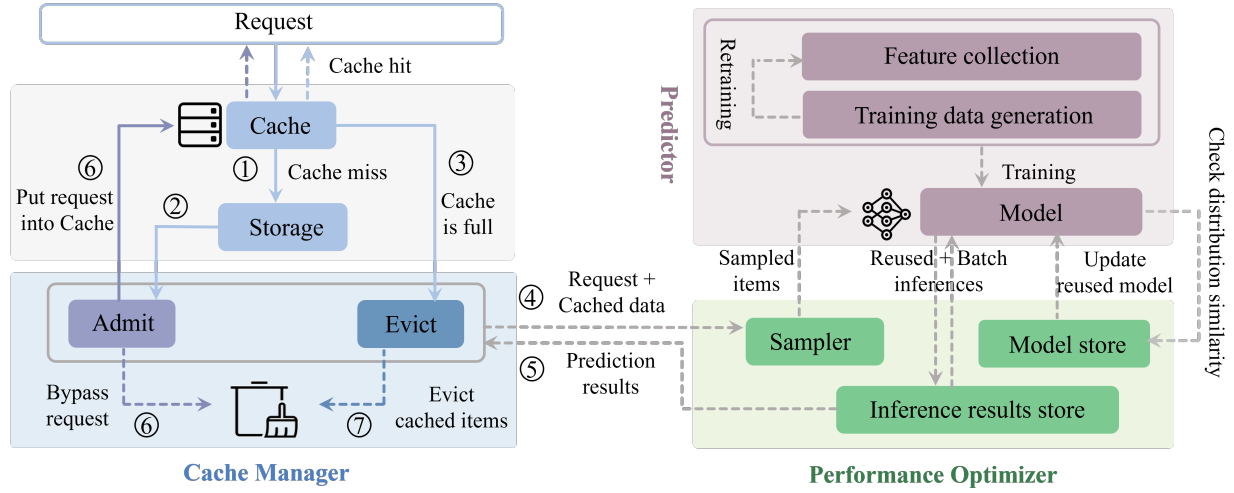


Fig. 1. An overview of the $LBSC^+$ framework.

BeladySizeCost. The functionality of each component is as follows:

Cache Manager governs online decision-making and coordinates admission and eviction. When a new request arrives, this component determines whether it should be admitted into the cache and identifies which cached item(s) should be evicted once the cache exceeds its capacity based on the prediction results. The complete process will be detailed in Section III-B.

Predictor serves as the learning component of $LBSC^+$. It continuously generates training data based on access histories, and learns from BeladySizeCost using supervised signals. After training, it predicts the next-reuse distances of cached items, from which $LBSC^+$ computes cost-aware utility scores in Eq. 3 to jointly guide admission and eviction. The model aims to capture varying data-access patterns in cloud environments. We will detail the model training and inference in Section III-C.

Performance Optimizer aims to minimize the computational overhead of the learning-based model. It employs several lightweight yet effective techniques, including adaptive inference reusing, adaptive sampling, and model reusing. This component enables $LBSC^+$ to maintain high prediction accuracy while significantly reducing CPU consumption during online decision-making. We will elaborate on these optimizations in Section IV.

B. Cache Manager

This component coordinates the end-to-end workflow of $LBSC^+$, encompassing both admission and eviction decisions under dynamic cloud workloads. Prior caching algorithms typically focus exclusively on eviction, removing items after the cache becomes full while admitting every new request unconditionally [5], [13], [20]. This strategy underutilizes the cache space: if an incoming item has a low access probability in the future, caching it brings little benefit and may trigger unnecessary evictions. A unified treatment of admission and eviction is therefore essential to ensure that only high-utility items occupy limited cache space. Preserving this motivation,

$LBSC^+$ integrates both decisions into a single workflow guided by a shared model with the same learning objective.

The detailed workflow of the cache manager is illustrated in Algorithm 1. When a new request α_t arrives, the $LBSC^+$ framework first updates features of α_t to assist training data generation (as described later in Section III-C), and the generated training data is put into D_t (lines 4-6). Then, $LBSC^+$ checks whether the requested data item already exists in the cache. If a cache hit occurs, the cached item is then returned to the user (lines 7-8). If a cache miss occurs, the framework enters the joint admission-eviction stage.

In this stage, if the cache is not full, the new request α_t is directly inserted into the cache, after which the algorithm proceeds to process the next request (lines 10-13). Otherwise, the metadata of the new request α_t is inserted into a temporary candidate pool G_{tmp} along with the metadata of all cached items (line 14). This pool forms the set of items jointly considered for admission and eviction. Noting that the pool occupies little space, as it maintains only metadata, not the actual data content. $LBSC^+$ then iteratively restores the cache to its capacity limit. During each iteration (lines 15-21), the system applies the inference-reuse mechanism to avoid re-computing predictions for items whose prior inference results are still reliable. An adaptive sampling is then performed using a splitting strategy, which reduces the number of model inferences by focusing inference on items with cost-to-size ratios. Here, P denotes the reused item predictions, C the cached items, and S the sampled items. The details of the inference-reuse mechanism and the adaptive sampling are described in Sections IV-A and IV-B, respectively.

In line 18 of Algorithm 1, the cache manager calls the predictor to perform batch model inference on the sampled items and uses the estimated utility scores, where each utility score is derived from the predicted next-reuse distance together with the item's fetching cost and size. Items with the lowest predicted utility are removed until the cache state satisfies the capacity constraint. That is, when the new request receives the lowest score, it is bypassed entirely by the cache manager.

Algorithm 1 $LBSC^+$

Input: Workload $\mathcal{R} = (\alpha_1, \alpha_2, \dots, \alpha_T)$, cached items C , and current prediction model M_t , set of prediction models $\mathcal{M}$, model update trigger threshold δ_D , and JS-divergence threshold T_{JS}

Output: Updated cached items C

```

1:  $D_t = \text{list}()$ 
2: Start background Model Refresh( $D_t, \mathcal{M}, \delta_D, T_{JS}$ )
3: for each incoming request  $\alpha_t$  do
4:   update_features( $\alpha_t$ )
5:    $d_t = \text{generate\_training\_data}(\alpha_t)$ 
6:    $D_t.add(d_t)$ 
7:   if  $\alpha_t$  is in cache then
8:     continue
9:   else
10:    if not is_cache_full( $\alpha_t$ ) then
11:      put  $\alpha_t$  into the cache
12:    continue
13:    end if
14:     $G_{tmp} = \text{metadata}(C \cup \{\alpha_t\})$ 
15:    while is_cache_full( $\alpha_t$ ) do
16:       $P = \text{inference\_reuse}(G_{tmp})$ 
17:       $S = \text{adaptive\_sample}(C)$ 
18:       $U = \text{batch\_model\_inference}(S \cup \alpha_t, M_t)$ 
19:       $e = \text{joint\_admission\_eviction}(G_{tmp}, P, U)$ 
20:      if  $e = \alpha_t$  then
21:        bypass  $\alpha_t$ 
22:      else
23:        apply_eviction( $e$ )
24:      end if
25:    end while
26:  end if
27: end for
28: return  $C$ 

```

Otherwise, one or more cached items are evicted, and the new request is inserted into the cache.

When the count of generated training data in D_t is larger than a predefined threshold δ_D , $LBSC^+$ would check whether the current workload distribution has shifted to invoke model retraining, as shown in Algorithm 2. The Jensen–Shannon (JS) divergence [23] between the most recent training batch and all previously stored batches is compared here, and the retraining of the model occurs only if every divergence exceeds the threshold T_{JS} . Otherwise, the most similar model is reused, thereby avoiding unnecessary training overhead (lines 3–8). The updated model is denoted as M_t in the algorithm, and this process runs asynchronously in a background to avoid blocking the processing of requests (line 2 in Algorithm 1). The details on this process are elaborated in Section IV-C.

Importantly, in $LBSC^+$, both admission and eviction decisions share the same model because their objectives are inherently consistent: both aim to maximize the overall cost-benefit of the cached items under dynamic workloads. Using a unified model ensures that admission and eviction are guided by the same learned utility function.

Algorithm 2 ModelRefresh

Input: Training dataset D_t , set of prediction models $\mathcal{M}$, current prediction model M_t , model update trigger threshold δ_D , and JS-divergence threshold T_{JS}

Output: Updated prediction model M_t

```

1: while The request stream is active do
2:   if  $\text{len}(D_t) \geq \delta_D$  then
3:      $B_t = \text{sample\_recent\_batch}(D_t)$ 
4:     if  $\text{calculate\_min\_JS}(B_t) > T_{JS}$  then
5:        $M_t = \text{retrain}(D_t)$ 
6:        $\mathcal{M} = \mathcal{M} \cup M_t$ 
7:     else
8:        $M_t = \text{find\_similar\_model}(B_t, \mathcal{M})$ 
9:     end if
10:  end if
11: end while
12: return  $M_t$ 

```

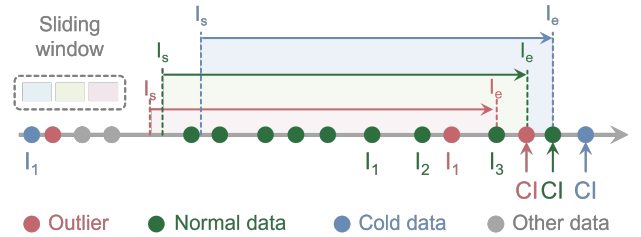


Fig. 2. Generation of three types of training data using a sliding window.

C. Predictor

This learning component is responsible for training and maintaining the shared model that supports joint admission and eviction decisions. It encompasses three major stages: feature collection, model training, and model inference. In addition, an outlier filtering mechanism is integrated to enhance training stability and accuracy.

1) *Feature Collection*: Since features play a crucial role in machine learning [24], we carefully select the following features, inspired by the recent state of the arts [5], [13].

Frequency. The number of re-accesses for each item.

Delta. The number of accesses between two consecutive requests of the same item. Specifically, Δ_1 denotes the number of accesses since the most recent request, Δ_2 denotes the number of accesses between the two previous requests of that item, and so forth. Figure 2 illustrates an example, where the items colored green correspond to repeated accesses of the same item. In this case, $\Delta_1 = I_3 - I_2$ and $\Delta_2 = I_2 - I_1$, where I_1, I_2, I_3 denote the request indexes.

Domain Features. To customize the cache for different systems, domain knowledge is incorporated as auxiliary features to strengthen the learning model. For instance, in cloud databases, the cached items have varying sizes, and each item has its own metadata, including table name, column name, etc.

2) *Model Training*: We first introduce how to generate labels for the training data. Once a new request arrives, a

sliding window¹ is employed to generate the corresponding label. Figure 2 shows the concept of sliding window $[I_s, I_e]$. The length (or size) of the sliding window represents the maximum number of requests it covers, i.e., $I_e - I_s$, denoted as L_e^s for simplicity. Initially, when the number of requests exceeds L_e^s , we start generating training data for each new request. After generation, the sliding window moves forward so that I_e points to the current request (i.e., CI shown in Figure 2). Additionally, we maintain a separate history list to retain features of the items that appear in the sliding window but do not exist in the cache. Therefore, if the requested item is in the history list or the cache, the difference between the current index and its last access index becomes the label. Otherwise, if the item's last request falls outside the sliding window, it is labeled as cold data and assigned a large value ($2 \cdot L_e^s$ in our implementation) to indicate infrequent access. For instance, in Figure 2, the label of the data colored green is " $CI - I_3$ " when a new request comes. Additionally, the last access of the item colored blue, i.e., I_1 , exceeds the L_e^s , so we label it as " $2 \cdot L_e^s$ ". After labeling, the sliding window moves forward to generate the next training data. To further improve the model performance, we propose a simple yet efficient method to filter the outliers in the training data.

Outlier detection on training data. We continuously generate new training data, yet not every data is effective for the model. Such data are outliers and can be pruned to further improve the model performance. For example, in Figure 2, we observe that the red-colored data is not accessed for a long time before I_1 , indicating that it belongs to "cold" data. However, we erroneously label it as $CI - I_1$, which suggests that it will be accessed after a short time. We formally define the outlier detection procedure as follows:

Definition 1. Outlier Detection. Given L_e^s , the training data D^i with label y^i and corresponding last reuse Δt_1^i . For every generated training data D^i , it is an outlier if: $y^i < L_e^s$ and $\Delta t_1^i > L_e^s$. When we detect the training data as an outlier, we remove it from the training data set.

After generating sufficient training data, we proceed to train a prediction model. $LBSC^+$ adopts gradient boosting machines (GBM) [26], since tree models do not require feature normalization and support concurrent training, thereby reducing CPU resource consumption. In addition, GBM has demonstrated strong effectiveness in prior works [13], [27]. To adapt to evolving workloads, $LBSC^+$ retrain the model by default once enough new training data has been collected. However, continuous retraining may introduce unnecessary computation overhead. To address this issue, we propose a model reuse strategy, which is described in Section IV-C.

3) **Model Inference:** As noted in Section III-B, when the cache reaches its capacity limit, $LBSC^+$ performs model inference to determine whether to admit the new item into the cache and which cached items to evict. Since the major computation overhead comes from the inferences of the newly requested data and cached data items, we employ several

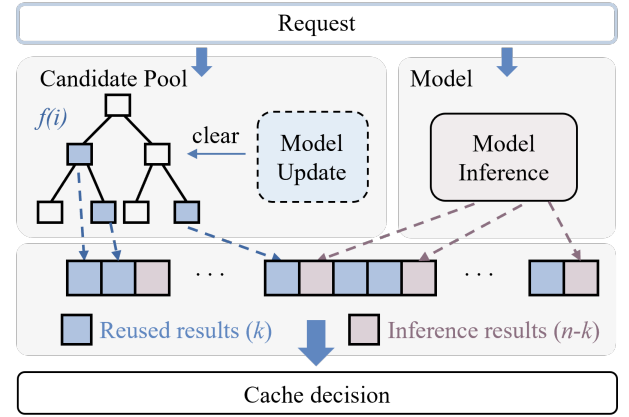


Fig. 3. The adaptive inference reusing process of $LBSC^+$.

optimizations to reduce the total number of model inferences. First, we introduce an adaptive inference-reusing mechanism that dynamically determines which previous predictions to reuse across multiple model inferences. We use a red-black tree [28] to store previous prediction results, allowing $LBSC^+$ to efficiently reuse them when triggering multiple evictions. Furthermore, we utilize a sampling-with-replacement strategy on the cache items before the model inference. The motivation of the sampling is mainly due to the large number of the caching items – as reported in [8], the cache usually consists of hundreds of thousands of items in production environments. After the sampling, we conduct the inference on the sampled items and the newly requested item, and select the items to evict based on the inference results. Finally, we find a specific property under the cost-aware setting, which allows us to further reduce the computation overhead when making model inferences. We illustrate the details in Section IV.

IV. PERFORMANCE OPTIMIZATIONS

In this section, we further present several optimizations applied during the training and inference phases, aiming to reduce the computational overhead of $LBSC^+$.

A. Inference reusing

In learning-based caching systems, model inference is invoked frequently to support admission and eviction decisions, and often dominates the overall computation overhead. However, across consecutive cache decisions, the majority of cached items remain unchanged, and their relative utility ordering does not immediately become stale. This temporal stability creates an opportunity to reuse previously computed inference results instead of recomputing them at every decision point. Nonetheless, directly reusing a fixed number of predictions per decision may cause limited robustness under dynamic workloads. Excessive reuse may apply stale predictions and degrade cache decisions, while conservative reuse forfeits potential efficiency gains. Therefore, an effective reuse mechanism must adaptively balance prediction freshness and inference reduction.

$LBSC^+$ introduces an adaptive inference reusing mechanism that selectively reuses prior predictions based on their

¹The concept of sliding window has been proposed in previous works [13], [16], [25]. We refer interested readers to these works for the impact of sliding window size on model accuracy and efficiency.

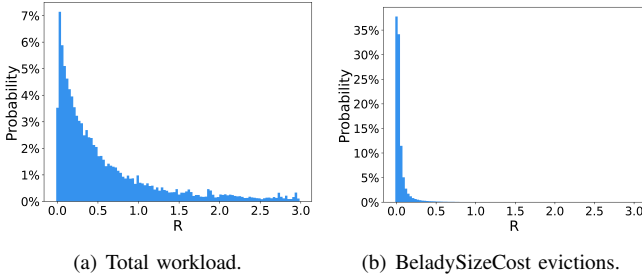


Fig. 4. Distribution of $\mathcal{R}$ between the total workload and BeladySizeCost evictions.

estimated freshness. The key idea is to maintain a candidate pool $\mathcal{P}$ that stores predicted utility scores of cached items from previous decisions, and to determine reuse eligibility using a freshness metric defined as:

$$f(i) = \frac{score_i}{\log(1 + pred_i)} \quad (16)$$

where $score_i$ is the model's predicted score for item i , and $pred_i$ denotes the elapsed time since item i 's last model inference, indicating how stale its previous prediction has become. The logarithmic term normalizes temporal staleness, stabilizing the scale of the freshness metric under varying reuse intervals. A larger $f(i)$ indicates that the prediction remains reliable and can be safely reused.

With the freshness metric, $LBSC^+$ performs inference reusing as illustrated in Figure 3: when a new request arrives, $LBSC^+$ first computes the freshness score $f(i)$ for each item in the candidate pool $\mathcal{P}$ and determines how many predictions can be reused. Specifically, the items in $\mathcal{P}$ are sorted in descending order of their freshness scores $f(i)$, and items with $f(i) > \tau$ are selected for reuse across consecutive decisions, where τ is a user-defined threshold. During this process, previously reused predictions are applied directly to admission and eviction decisions without invoking the model. Only the remaining items require fresh inference, thereby reducing the total inference count. Whenever the model is retrained, all cached predictions in $\mathcal{P}$ are invalidated to ensure consistency. In addition, if any item in $\mathcal{P}$ is accessed again, its score and freshness must be immediately updated. To efficiently manage freshness metadata and reuse decisions, $LBSC^+$ employs a red-black tree structure indexed by $f(i)$. When triggering multiple evictions, the structure enables fast lookup in $O(\log n)$ and incremental updating of low-utility items.

B. Sampling with cost-size threshold

To further reduce the number of samples for model inferences, narrowing the scope of sampling is a reasonable way. We first introduce the ratio of cost to the size of every item, denoted as $\mathcal{R}$, as a metric. Formally, for every item i , we define the ratio R_i as follows:

$$\mathcal{R}_i = \frac{c_i}{s_i}. \quad (17)$$

Figure 4 shows the distribution of $\mathcal{R}$ for a representative workload (see Section VI-A), along with the corresponding evictions made by BeladySizeCost. Although $\mathcal{R}$ spans a wide range across the entire workload (from 0.0 to 3.0 in Figure 4(a)), BeladySizeCost predominantly evicts items with relatively small $\mathcal{R}$ values. As illustrated in Figure 4(b), more than 99% of the evicted items have $\mathcal{R}$ between 0.0 and 1.0. This observation is consistent with prior works [29], [30]: under typical workloads, items with higher $\mathcal{R}$ (i.e., higher cost per byte) are more likely to be retained in the cache. Therefore, when model inference is required for cache decisions, it is desirable to only sample items with lower $\mathcal{R}$ values, which can further reduce inference overhead. Based on this insight, we propose the following two methods to narrow the sampling range.

Static Splitting Method (SSM). We partition the cached data items into two groups according to a predefined cost-size ratio threshold (denoted as $T_{\mathcal{R}}$). Sampling is then performed exclusively from the subset whose $\mathcal{R}$ values are lower than $T_{\mathcal{R}}$. We formally define *SSM* in Definition 2, where *baseline* denotes sampling from the entire cached items while keeping other settings unchanged.

Definition 2. Given a threshold $T_{\mathcal{R}}$ and a cache containing N_C items, the baseline samples $\mathcal{U}_{ori}$ items whenever the cache is full. We partition the cached items into two disjoint groups $\mathcal{R}^l$ and $\mathcal{R}^h$, such that:

$$\begin{aligned} \mathcal{R}_i &< T_{\mathcal{R}}, \forall i \in \mathcal{R}^l \\ \mathcal{R}_j &\geq T_{\mathcal{R}}, \forall j \in \mathcal{R}^h \end{aligned} \quad (18)$$

Under the *SSM* scheme, only $\frac{Num(\mathcal{R}^l) \cdot \mathcal{U}_{ori}}{N_C}$ items are sampled from $\mathcal{R}^l$ whenever the cache reaches capacity, while the total cost $\mathcal{C}$ of *SSM* still (approximately) converges to the baseline. Here, $Num(\mathcal{R}^l)$ indicates the number of items contained in $\mathcal{R}^l$.

The *SSM* method aims to keep the total cost $\mathcal{C}$ close to that of the *baseline*. Therefore, the choice of $T_{\mathcal{R}}$ is crucial. If $T_{\mathcal{R}}$ is set too small, some items that should be evicted may never be sampled, which can lead to an increase in the total cost $\mathcal{C}$. In contrast, if $T_{\mathcal{R}}$ is too large, $\mathcal{R}^l$ will include more items, limiting the reduction in computation overhead. Consequently, the effectiveness of this method depends on the prior knowledge of $\mathcal{R}$'s distribution.

Dynamic splitting method (DSM). In production environments, the distribution of $\mathcal{R}$ over the entire workload is typically unknown, which makes the *SSM* difficult to apply. To address this limitation, we propose a dynamic splitting method (*DSM*), as outlined in Algorithm 3. At the beginning of the workload, *DSM* samples items proportionally from both $\mathcal{R}^l$ and $\mathcal{R}^h$, and increase the threshold $T_{\mathcal{R}}$ whenever the evicted item's ratio $\mathcal{R}$ (denoted as $E_{\mathcal{R}}$) exceeds $T_{\mathcal{R}}$ (lines 3-9). This design avoids the adverse impact of an overly small initial $T_{\mathcal{R}}$ on the cost $\mathcal{C}$. If, over a sustained period, all evicted items have $\mathcal{R} < T_{\mathcal{R}}$ (i.e., when $\hat{T}_1 \geq T_1$), *DSM* assumes that restricting sampling to $\mathcal{R}^l$ will not noticeably affect the total cost $\mathcal{C}$. Accordingly, the algorithm switches to sampling exclusively from $\mathcal{R}^l$ using the same sampling budget as in

Algorithm 3 Dynamic Splitting Method

Input: Cached items C , splitting threshold $T_{\mathcal{R}}$, original sample size $\mathcal{U}_{ori}$, T_1 , and T_2 , DSM state $(\hat{T}_1, \hat{T}_2)$

Output: Sampled items $\mathcal{S}$ and updated DSM state

- 1: $\mathcal{S} = \emptyset, E_{\mathcal{R}}$ is the ratio $\mathcal{R}$ of the evicted item
- 2: **if** $\hat{T}_1 < T_1$ **then**
- 3: Sample $\frac{Num(\mathcal{R}^l) \cdot \mathcal{U}_{ori}}{N_C}$ items from $\mathcal{R}^l$, put into $\mathcal{S}$
- 4: Sample $\frac{Num(\mathcal{R}^h) \cdot \mathcal{U}_{ori}}{N_C}$ items from $\mathcal{R}^h$, put into $\mathcal{S}$
- 5: **if** $E_{\mathcal{R}} > T_{\mathcal{R}}$ **then**
- 6: $T_{\mathcal{R}} = T_{\mathcal{R}} + 0.1, \hat{T}_1 = 0$
- 7: **else**
- 8: $\hat{T}_1 = \hat{T}_1 + 1$
- 9: **end if**
- 10: **else**
- 11: Sample from $\mathcal{R}^l$ only, and $\hat{T}_2 = \hat{T}_2 + 1$
- 12: **if** $\hat{T}_2 > T_2$ **then**
- 13: $\hat{T}_2 = 0$
- 14: Sample from $\mathcal{R}^l$ and $\mathcal{R}^h$ separately, put into $\mathcal{S}$
- 15: **if** $E_{\mathcal{R}} > T_{\mathcal{R}}$ **then**
- 16: $T_{\mathcal{R}} = T_{\mathcal{R}} + 0.1$
- 17: **end if**
- 18: **end if**
- 19: **end if**
- 20: **return** $\mathcal{S}, (\hat{T}_1, \hat{T}_2)$

SSM (line 11). Since the distribution of $\mathcal{R}$ may evolve over time, *DSM* periodically resumes sampling from the entire cache after a fixed interval T_2 . During this phase, $T_{\mathcal{R}}$ is further increased whenever an evicted item's $\mathcal{R}$ exceeds $T_{\mathcal{R}}$ (lines 12–18). Notably, $T_{\mathcal{R}}$ monotonically increases in our algorithm, ensuring robust performance w.r.t. the total cost $\mathcal{C}$. This is because our method can hold $\mathcal{R}^l$ with almost all data that should be evicted.

C. Model reusing

Although continuous retraining enables the model to adapt to evolving workloads, doing so at high frequency is often unnecessary. When the distribution of the current training batch closely resembles that of a previous batch, the corresponding model can be reused to effectively reduce retraining overhead. In this work, we define a batch as a sequence of consecutively generated training data whose count equals a predefined batch size².

Prior research [16] has shown that the access patterns in production workloads typically follow a standard Zipfian distribution. Accordingly, we represent the access patterns of different batches as standard Zipfian distributions. To quantify the similarity of two batches, i.e., the similarity between their underlying distributions, we adopt the Jensen-Shannon (JS) divergence [31], defined as follows:

Definition 3. The JS-divergence distance between two probability distributions $\vec{p} = (p_1, \dots, p_n)^T \geq 0$ and $\vec{q} = (q_1, \dots, q_n)^T \geq 0$ is defined as:

²We leave the research to better split the training data into batches and set a proper batch size for our future work.

$$JS(\vec{p}, \vec{q}) = \frac{1}{2} (KL(\vec{p} \parallel \frac{\vec{p} + \vec{q}}{2}) + KL(\vec{q} \parallel \frac{\vec{p} + \vec{q}}{2})), \quad (19)$$

where

$$KL(\vec{p} \parallel \vec{q}) = \sum_{i=1}^n p_i \log(\frac{p_i}{q_i}). \quad (20)$$

The Zipfian distribution is characterized by $p_i = A/i^\alpha$, where A denotes the normalization constant and α is the Zipfian parameter. In practice, these parameters can be estimated using a Least-Squared-based approach [16]. However, this estimation procedure incurs extra computation overhead and may yield inaccurate parameter values. To avoid these issues, our implementation directly samples training items together with their access frequencies for subsequent computation.

Overall, when model retraining is required, we first sample a subset of items from the current batch and compute their JS divergence with respect to previous batches. If the JS divergence between the current batch and all historical batches exceeds a predefined threshold (i.e., T_{JS} in Algorithm 2), we retrain the model and store it. Otherwise, we reuse the previously trained model associated with the minimum JS divergence. More details can be found in our technical report [32].

V. INTEGRATION WITH CLOUD DATABASES

LBSC⁺ is designed as a modular caching framework that can be integrated into a wide range of cloud database systems with minimal architectural changes. In this section, we first demonstrate a concrete integration with FPDB to illustrate the end-to-end deployment workflow. We then discuss how *LBSC⁺* can be extended to commercial cloud databases and the practical considerations involved.

A. Integration with FPDB

FPDB [7] is a cloud analytical database that adopts a storage-disaggregation architecture. It stores original data on the storage node using AWS S3 [9] or MinIO [33]. FPDB supports predicate push-down to reduce network traffic, and caches intermediate query results, referred to as segments, at compute nodes to mitigate data-fetching overhead. Each segment corresponds to a column within a table partition, and segment sizes vary depending on schema and query patterns.

The default cache algorithm in FPDB is called WLFU. It assigns weights to each segment based on transfer and computation costs and evicts the one with the lowest weight. We replace WLFU with *LBSC⁺* at the segment cache layer. Since *LBSC⁺* operates on cache items with associated costs and features, this replacement does not require changes to FPDB's query execution engine.

A key challenge in FPDB integration is that the effective cost of a segment may change across accesses, as different queries may consume only a subset of the segment. To address this, we dynamically estimate the effective cost of a segment as follows:

$$\mathcal{C}_{new} = \frac{\mathcal{C}_{seg} + \mathcal{C}_{old} \cdot (num - 1)}{num}. \quad (21)$$

TABLE II
DATASETS USED IN THE EVALUATION.

Dataset	Wiki ₁	Wiki ₂	SSB
Unique Items (Million)	11.1	7.43	0.1
Total Requests (Million)	100	100	20
Total Item Size (GB)	1071	854.95	2172
Max Item Size (MB)	557.89	674.25	209
Min Item Size (Byte)	42	10	1
Mean Item Size (MB)	0.09	0.12	21.72

where $\mathcal{C}_{seg}$ is the cost of the current access, $\mathcal{C}_{old}$ is the previous estimation, and num denotes the number of accesses. Intuitively, the estimated cost of an item is calculated as the average of its history data fetching costs, which yields accurate cost estimation with negligible runtime overhead, as validated in Section VI.

B. Extension to Commercial Cloud Databases

The modular design of $LBSC^+$ enables its integration into mainstream cloud databases such as OceanBase [34], TiDB [35], or PolarDB [36]. However, several practical challenges must be addressed for effective deployment:

Heterogeneous Caching Granularity. Commercial cloud databases differ in what they cache: some systems store final query results, while others cache intermediate results or raw table data. This heterogeneity affects both feature extraction and cost modeling. To ensure accurate predictions, $LBSC^+$ must adapt its cost definition and feature set to match the caching granularity of the target system – just like what we have done in the integration with FPDB.

Feature Collection Overheads. Some systems do not explicitly maintain fine-grained access metadata required by the model (e.g., per-item hit frequency or cost). In such cases, it is necessary to design lightweight feature extraction methods that capture the required features while minimizing the additional overhead introduced.

Overall, integrating $LBSC^+$ into commercial cloud databases requires modest architectural changes, preserving compatibility with existing components. Once deployed, $LBSC^+$ can provide an adaptive, cost-aware caching capability, effectively improving throughput and reducing latency in real-world cloud environments.

VI. EVALUATION

In this section, we present the experimental evaluation of $LBSC^+$ and compare its performance against state-of-the-art baselines. Our code is publicly available at [37].

A. Experimental Setup

Testbed: We evaluate on Alibaba Cloud ECS [38] with real cloud database deployments. Compute and storage nodes are provisioned with 16 vCPUs and 64 GB of memory, and they communicate over a 5Gbps network. All servers run on the Ubuntu 20.04 operating system. For experiments using synthetic datasets, we use a single dedicated server with an Intel Xeon W-2275 3.30GHz 14-core CPUs and 512 GB

of main memory. Additionally, our experiments use FPDB (commit e439414) with MinIO as the storage backend, and OceanBase version 4.5.0.0.

Datasets and Benchmark: We evaluate $LBSC^+$ on synthetic traces and real cloud databases, and the key statistics of these datasets are summarized in Table II. For experiments using synthetic datasets, we use a single dedicated server with an Intel Xeon W-2275 3.30GHz 14-core CPUs and 512 GB of main memory.

Synthetic Datasets. We first use two real-world traces [13] to evaluate the performance of $LBSC^+$ in a simulated environment. Both traces are collected from a west-coast node that serves photos and other media content for Wikipedia pages. To simulate the cloud environment, we generate the data cost for the items from the original datasets, including the computation cost and transfer cost mentioned in Eq. 1. In detail, we rewrite the transfer cost $\mathcal{TC}_i$ as $k \cdot s_i$ for each data item i . Recall that k presents network transmission speed calculated by real network bandwidths and s_i denotes the size of the data item i . As for the computation cost $\mathcal{CC}_i$, we uniformly sample from a predefined range and assign $\mathcal{CC}_i$ according to the sampling results. In our experiments, we conduct thorough comparisons under different settings and signify the specific settings in each use case, and we use the first 6,710,886 queries for caching warm-up.

Star Schema Benchmark (SSB) [39]. This benchmark is a variant of TPC-H [40], which consists of one fact table and four dimension tables, and naturally exhibits skewed access patterns and heterogeneous data-fetching costs. SQL workloads are generated from predefined SSB query templates. Predicate parameters are instantiated following Zipfian distributions [41], and the first 50 queries are used for caching warm-up. To construct the changing-workload trace, we sequentially concatenate seven types of queries with skew factors θ ranging from 0.0 to 2.0. Each query type contains 3,900 queries. Since the seven query types are concatenated sequentially, all warm-up queries come from the first query type.

Implementation. We integrate $LBSC^+$ in FPDB [7] and OceanBase [34] for end-to-end system evaluation using SSB. For FPDB, cached units correspond to intermediate query results, and $LBSC^+$ is integrated into FPDB's segment cache layer. We construct analytical queries involving multiple joins and aggregations, and each cache item is associated with the data-fetching cost of the corresponding intermediate result. For OceanBase, we adopt a non-intrusive design by attaching an external cache layer that stores the final query results. Under this setting, we generate relatively simple SQL queries with short per-query execution times. The cost of each cache item is defined as the end-to-end query execution latency.

Baselines: We compare $LBSC^+$ against a set of baselines, including both heuristic-based caching algorithms and machine learning-based methods. These baselines are described as follows.

- *Belady* [14] evicts the item with the furthest access.
- *LFUDA* [42] is an extension of LRU [43], where an item's priority key is determined by its frequency plus the cache

age. Note that LFU evicts the item with the least access frequency.

- **LRUK** [6] is an extension of LRU [6] that utilizes k LRU stacks. Note that LRU evicts the item that is the least recently used.
- **GDSF** [29] ranks the cached data by $rank_i = \frac{f_i \cdot C_i}{s_i} + L_i$, where f_i represents access frequency, C_i is the *total cost*, L_i is the rank of the last evicted item. In FPDB, the caching algorithm WLFU is similar to GDSF, but it does not consider the L_i in its ranking function.
- **LHD** [20] models the hit density based on age and size, and evicts the item with the lowest hit density.
- **CACHEUS** [5] employs a regret-minimization framework [44] to dynamically choose between two expert eviction policies on each cache miss. Since the original algorithm assumes uniform-sized objects, we re-implement and adapt it to support variable-size data.
- **LRB** [13] is a learning-based caching algorithm that mimics the offline Belady algorithm. It uses an ML model to predict the next request time of the cached data.
- **LBSC** [11] is a cost-aware learning-based caching framework, which optimizes only eviction while admitting all new items by default. Moreover, it does not incorporate the inference reusing strategy.

Metrics: Our main metric is the total cost $\mathcal{C}$ defined in Section II-A. Due to the diversity of the workloads, the total cost $\mathcal{C}$ can vary widely. Hence, to facilitate visual presentation, we calculate the $\mathcal{C}$ values of every caching algorithm and report them compared to GDSF (for its popularity with cost-aware consideration) using the metric: $\frac{\mathcal{C}_{GDSF} - \mathcal{C}_{algorithm}}{\mathcal{C}_{GDSF}}$. Each experiment starts with a cache warm-up phase where no metrics are recorded. For the SSB datasets, the metric is the end-to-end execution time of all queries. Furthermore, the metric in Section VI-B3 is the computation overhead (or cost) reduction compared to the *baseline*, which indicates $LBSC^+$ without the corresponding optimization in Section IV.

B. Performance Evaluation

In this subsection, we evaluate the effectiveness of $LBSC^+$ from three perspectives. First, we compare its overall performance against state-of-the-art caching algorithms using synthetic datasets under diverse cost settings and cache sizes. Second, we examine its effectiveness when integrated into real cloud database systems, namely FPDB and OceanBase. Finally, we analyze the impact of the proposed optimization techniques on computational overhead.

1) *Performance on Synthetic Datasets:* We evaluate $LBSC^+$ on two synthetic workloads derived from real-world traces under varying cache sizes and cost configurations. For each item, the total cost $\mathcal{C}$ is generated according to Eq. 1, and we vary the relative dominance of computation cost and transfer cost to emulate different cloud environments.

Figure 5 presents the cost reduction of $LBSC^+$ and seven baselines relative to GDSF under different cache sizes. Across all traces and cache sizes, $LBSC^+$ consistently achieves the lowest total cost, outperforming all baselines. Overall, $LBSC^+$ reduces the total cost by 10%-17% compared to

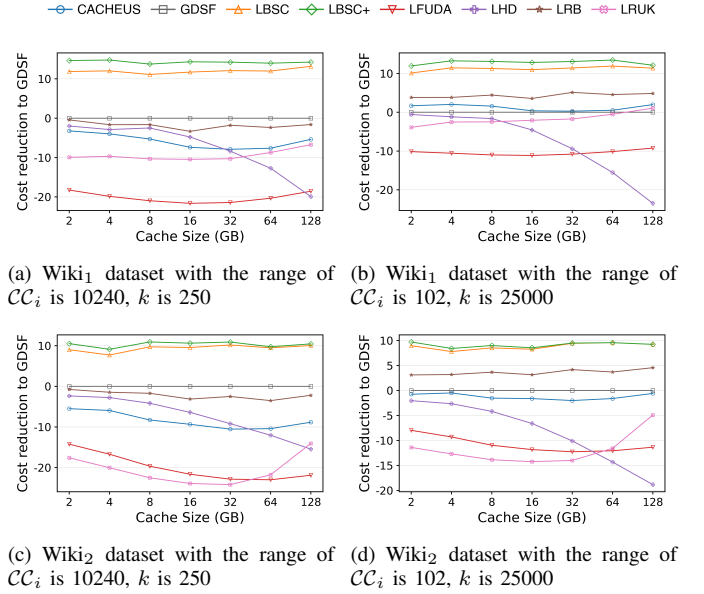


Fig. 5. The cost reduction to GDSF w.r.t varying cache sizes for $LBSC^+$ and the seven baselines on two datasets. On each dataset, the computation cost dominates in the sub-figures (a) and (c), and the transfer cost dominates in the sub-figures (b) and (d).

GDSF, regardless of whether computation cost in Figures 5(a) and 5(c) or transfer cost in Figures 5(b) and 5(d) dominates.

Figure 5 also highlights the impact of coordinated admission and eviction. Compared to LBSC, which admits all items by default, $LBSC^+$ achieves noticeably lower cost, especially under small cache sizes. This gap is most pronounced when cache capacity is limited (e.g., 2–16 GB in Figure 5), where unconditional admission in LBSC causes frequent eviction of high-utility items. As cache size increases, the performance gap between $LBSC^+$ and LBSC gradually narrows, indicating that admission control is particularly critical under tight memory constraints.

The results in Figure 5 reveal that, except for $LBSC^+$ and LBSC, no existing baseline consistently improves performance as cache size grows. When transfer cost dominates, the total cost trend closely follows byte miss rate, allowing LRB and CACHEUS to outperform other heuristics shown in Figures 5(b) and 5(d). In contrast, when computation cost dominates, byte miss rate becomes a poor proxy for total cost, and GDSF performs best among heuristics, evidenced in Figures 5(a) and 5(c). Nevertheless, due to limited adaptability to workload dynamics, all baselines remain inferior to $LBSC^+$ across configurations.

So far, we have demonstrated that $LBSC^+$ achieves significant improvements over state-of-the-art baselines. To evaluate the quality of the learning target, i.e., the BeladySizeCost algorithm proposed in Section II-B, we compare $LBSC^+$, LBSC, BeladySizeCost, BeladySizeCost without admission control, and the classical Belady algorithm in Figure 6. As shown in the figure, **BeladySizeCost consistently achieves the lowest total cost** across all workloads and cache sizes, validating its effectiveness as a cost-aware oracle. Moreover, BeladySizeCost clearly outperforms its admission-free variant,

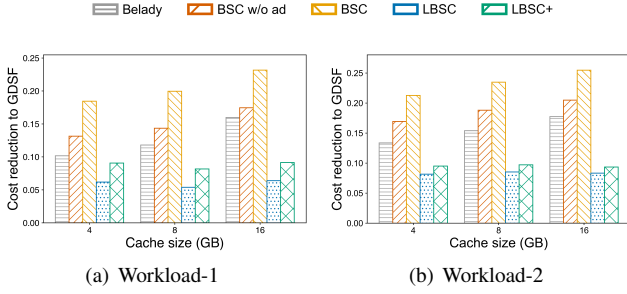


Fig. 6. *Total cost* comparison among Belady, BSC w/o ad, BSC, LBSC and $LBSC^+$. Note that “BSC” denotes BeladySizeCost, and “BSC w/o ad” means BeladySizeCost without admission control.

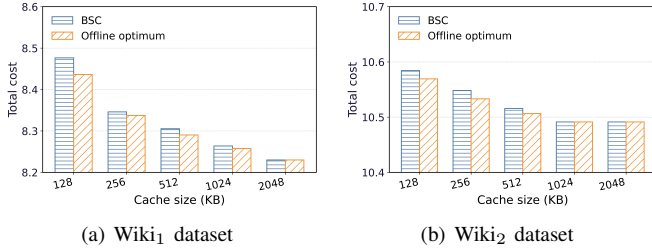


Fig. 7. Comparison between BeladySizeCost and the exact optimum under different cache sizes.

confirming that joint admission and eviction are essential while providing a strong oracle reference for cost-aware caching.

In contrast, the classical Belady algorithm performs substantially worse once heterogeneous data-fetching costs are introduced in Figure 6, demonstrating that it might be ill-suited as an oracle for cloud databases. Although a performance gap remains between $LBSC^+$ and BeladySizeCost, it is expected due to practical factors such as finite training data and constrained inference budgets. Despite these limitations, $LBSC^+$ remains superior to all learning-based and heuristic baselines.

Comparison with the Exact Optimum. To assess how well BeladySizeCost aligns with the global *total cost* objective, we compare it with the offline optimum. Given the complete workload, we use Integer Linear Programming (ILP) to derive the optimal caching decisions [17]. Since exact optimization is NP-hard and has exponential worst-case complexity [17], we use workload traces of 5,000 requests for this experiment. Figure 7 shows that BeladySizeCost achieves the *total cost* consistently close to the exact optimum across different cache sizes, demonstrating that the next-reuse approximation closely matches the global caching objective in practice.

Impact of Admission Control. We next evaluate whether the admission strategy of $LBSC^+$ can directly benefit existing caching algorithms. Both LRB and CACHEUS originally use unconditional admission, i.e., every missed item is admitted and their policies only determine subsequent evictions. We therefore incorporate the admission strategy of $LBSC^+$ into these two methods while keeping their original eviction policies unchanged. As shown in Figure 8, admission control reduces the *total cost* of LRB on both workloads by filtering

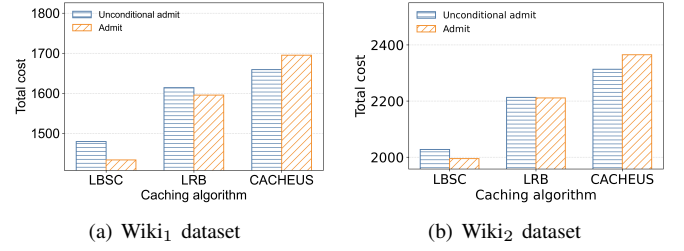


Fig. 8. Evaluation of incorporating the $LBSC^+$ admission strategy into different cache replacement policies on the Wiki₁ and Wiki₂ datasets.

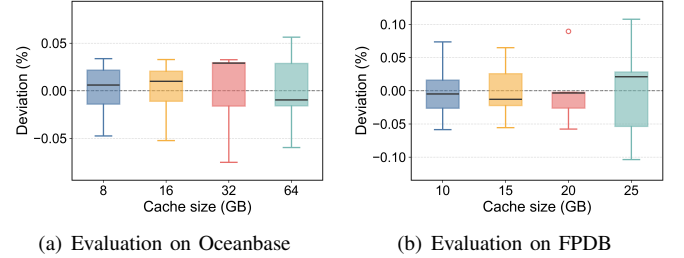


Fig. 9. Execution time deviations of $LBSC^+$ in OceanBase and FPDB over five repeated runs.

low-utility incoming items and alleviating cache pollution. In contrast, it does not improve CACHEUS, because bypassing incoming items changes the eviction-history information used by CACHEUS and can consequently affect its subsequent eviction decisions. These results indicate that admission control does not necessarily complement an independently designed eviction policy. In $LBSC^+$, admission and eviction are instead jointly determined by the same utility score, enabling coordinated cache decisions.

Robustness. We further evaluate the robustness of $LBSC^+$. First, we repeat the real-system experiments five times on both OceanBase and FPDB under different cache sizes. Since the execution time of the same query may vary slightly across runs because of system-level noise, Figure 9 reports the mean execution time and standard deviation. As shown in Figure 9, the execution-time deviations remain within 0.15% across all cache sizes in both systems, indicating that $LBSC^+$ is robust to moderate fluctuations in execution time. Second, since $LBSC^+$ randomly samples cached items for model inference, we repeat the experiments with five different random seeds and report the mean and standard deviation in Figure 10. Across both Wiki₁ and Wiki₂, the inference time and inference number exhibit only small variations, while the cost reduction remains consistently stable. These results demonstrate that $LBSC^+$ is robust to both system-level execution time variability and sampling randomness.

2) *Performance under FPDB and OceanBase:* We further evaluate $LBSC^+$ in real cloud database systems to demonstrate its practical effectiveness beyond synthetic traces. Specifically, we integrate $LBSC^+$ into FPDB and OceanBase and compare it with representative baseline caching policies. **FPDB.** Figure 11 compares $LBSC^+$ with LBSC, LFU and WLFU [7] in FPDB. Specifically, Figure 11(a) presents the

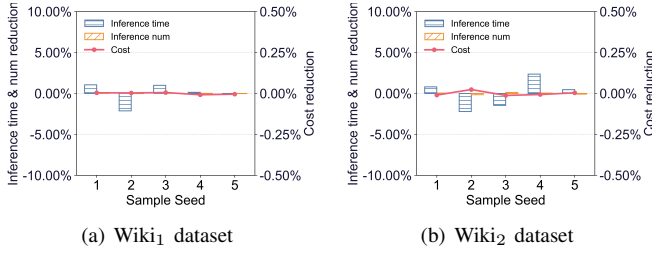
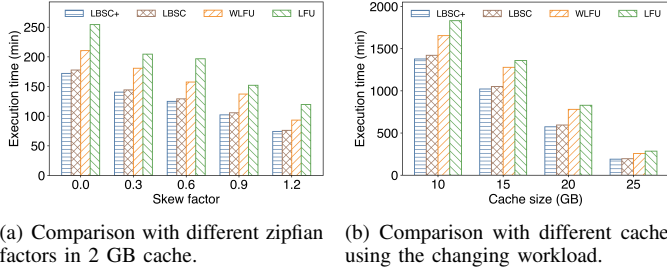


Fig. 10. Robustness to sampling randomness during model inference.

Fig. 11. End-to-end execution time comparison among $LBSC^+$, LBSC, WLFU, and LFU in FPDB.

execution time of five types of batch queries with a 2 GB cache. $LBSC^+$ consistently outperforms LBSC, LFU and WLFU. The most significant cost reduction occurs when $\theta = 0.3$, with $LBSC^+$ outperforming LFU and WLFU by 31.35% and 22.33%, respectively. Overall, $LBSC^+$ outperforms the WLFU algorithm by 21.26% on average in FPDB. Figure 11(b) illustrates the execution time of the changing workload with four different cache sizes. We can find that $LBSC^+$ performs best under changing workload, which demonstrates $LBSC^+$ can adapt well to the changing workload. For example, $LBSC^+$ outperforms WLFU by 26.40% when the cache size is 20 GB.

OceanBase. Figure 12 presents the evaluation results on OceanBase. Figure 12(a) shows the execution time of batch queries generated with different skew factors under an 8 GB cache. $LBSC^+$ consistently outperforms GDSF, LRB, and LBSC across all skew settings, highlighting its robustness to workload skew even when caching final query results. Figure 12(b) further examines the impact of cache size on a representative workload. Across all cache sizes, $LBSC^+$ achieves the lowest execution time, reducing execution time by up to 18% compared to GDSF and by 5% compared to LBSC. These results indicate that admission control and cost-aware learning remain beneficial even when the caching granularity shifts from intermediate results in FPDB to final query results in OceanBase.

Taken together, Figures 11 and 12 demonstrate that $LBSC^+$ delivers stable and superior performance in real cloud database systems. Its advantages persist across different caching granularities, workload skews, and cache sizes, underscoring the practicality of the proposed framework.

3) Optimizations on $LBSC^+$:

In this subsection, we evaluate the effectiveness of the three optimizations proposed in Section IV, i.e., adaptive inference

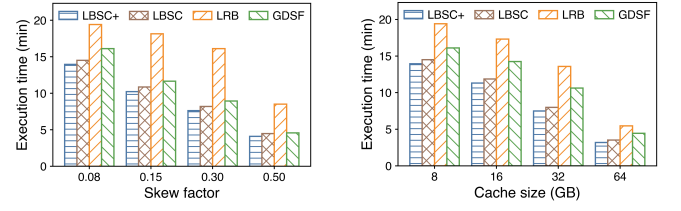
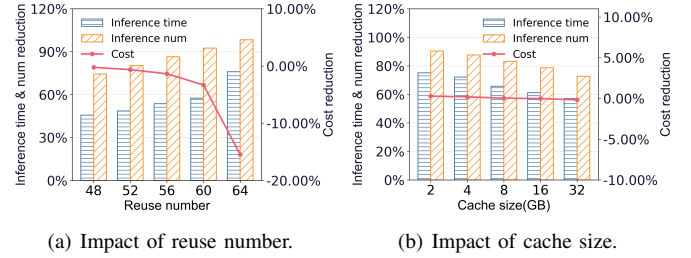
Fig. 12. End-to-end execution time comparison among $LBSC^+$, LBSC, LRB, and GDSF in OceanBase.

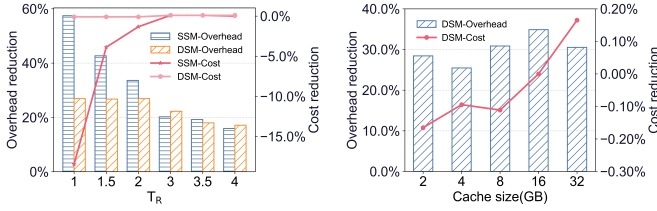
Fig. 13. In (a), the histogram shows the reductions in inference time and inference number compared to the *baseline* under different reuse numbers (i.e., static inference reusing), and the line chart illustrates the corresponding cost reduction. Note that the sample number is 64 by default. In (b), we evaluate the performance of our dynamic inference reusing strategy across different cache sizes.

reusing, sampling with cost-size threshold, and model reusing. Unless otherwise stated, all experiments use the same workload configuration as Figure 5(a), and the *baseline* refers to $LBSC^+$ without the corresponding optimization.

Inference Reusing. Figure 13 evaluates the effectiveness of the inference reusing strategy in Section IV-A.

We first examine the static reusing strategy, where a fixed number of previously computed predictions are reused across consecutive cache decisions. As shown in Figure 13(a), when the reuse number increases, both inference time and inference count are reduced, since fewer fresh model inferences are required. However, excessive reuse leads to stale predictions and degrades cache decisions. For instance, with a reuse number of 64, the cost increases by up to 15%, indicating that invalid reuse significantly harms cache decisions. On the other hand, when the reuse number is small, the reductions in inference time and inference number are marginal, resulting in limited improvement in computation overhead. Thus, the static reuse strategy is not robust, as determining an appropriate reuse number is sensitive to different workloads.

Figure 13(b) evaluates the dynamic reusing strategy, where it automatically determines the number of predictions to reuse at each decision based on the freshness metric defined in Eq. 16. Across different cache sizes, adaptive reuse consistently reduces the number of model inferences to about 85% and inference time to about 73% of the baseline, while maintaining comparable or slightly improved cost performance. Therefore, we observe that the dynamic strategy is robust across different cache sizes: when access patterns become stable, it reuses a larger portion of past predictions; when the



(a) Comparison between the *SSM* and *DSM* methods. (b) Comparison of *DSM* with varying cache sizes.

Fig. 14. In (a), the histogram presents the computation overhead reduction to the *baseline* w.r.t. different T_R values. The line chart shows the cost reduction to the *baseline* using *SSM* and *DSM*. In (b), we compare the *DSM* with varying cache sizes of a specific workload and the initial T_R is set to 0.5.

workload becomes volatile, the strategy automatically reduces the reuse number to refresh inference results.

Overall, Figure 13 demonstrates that adaptive inference reuse achieves a robust trade-off between computational efficiency and cache decision quality.

Sampling with Cost-size Threshold. Figure 14 compares the two sampling strategies: *SSM* and *DSM*.

Figure 14(a) shows that when the cost-size ratio threshold T_R is properly selected, *SSM* can substantially reduce computation overhead while preserving cost reduction close to the *baseline*. For example, when T_R is set to 3, the computation overhead reduction reaches about 20%, while the total cost remains nearly unchanged. This is because such a threshold already covers more than 99% of the evictions under *BeladySizeCost*, indicating that most evicted items lie in the subset $\mathcal{R}^l$. Therefore, restricting sampling to this subset maintains eviction quality while lowering inference overhead.

Figure 14(a) also illustrates a clear trade-off as T_R changes. A larger T_R expands the subset $\mathcal{R}^l$, but this weakens the overhead reduction effect. For instance, when T_R is set to 1, the computation overhead reduction exceeds 55%, but the total cost degrades by about 18%. When $T_R = 4$, the total cost is almost identical to the *baseline*, while the overhead reduction decreases to only 17%.

Figure 14(b) demonstrates that *DSM* is more robust when the workload distribution is unknown. Unlike *SSM*, which is sensitive to the initial threshold choice, *DSM* dynamically adjusts T_R according to observed eviction behavior. Across different cache sizes, *DSM* consistently achieves around 30% computation overhead reduction on average, while the corresponding cost loss remains below 0.7%. This indicates that *DSM* provides stable cost performance together with meaningful efficiency gains, without requiring distribution of the entire workload.

Model Reusing. Next, we evaluate the model reusing strategy in Section IV-C. In our experiments, we multiply all computed JS-divergence values by 10 before threshold comparison. The threshold is scaled consistently, which avoids comparisons among very small decimal values and reduces rounding effects without changing the relative ordering of the divergence values. Accordingly, the reported threshold $T_{JS} = 2.99$ corresponds to 0.299 on the original value. Figure 15(a) investigates the impact of the threshold on performance, including the total

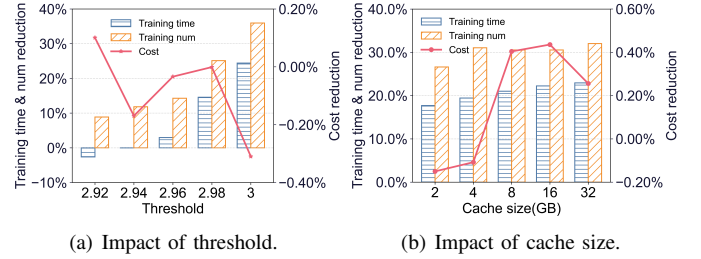


Fig. 15. In (a), the histogram presents the training time and the training number reduction to the *baseline* among different thresholds, and the line chart presents the corresponding cost reduction to the *baseline*. In (b), we compare the performance of model reusing at varying cache sizes, and the default similarity threshold is set as 2.99.

cost $\mathcal{C}$ and the computation overhead. We observe that the training time and the training number reduction both decrease when the threshold becomes small, but the corresponding cost reduction is increased. This is because the smaller the threshold, the fewer reusable models. Conversely, the reused model may not work well when the threshold is significantly big. Additionally, we find the training number reduction is consistently larger than the training time reduction. The reason is that computing JS divergence also produces extra overheads. When the threshold is set to 2.92, the training time reduction is even less than 0, indicating the overheads of calculating JS divergence cannot be negligible.

Figure 15(b) depicts the performance of model reusing w.r.t. varying cache sizes. We find that the training time reduction is around 20%, the training number reduction is around 30%, and the corresponding cost reduction is even by up to 0.4% when the cache size is 16 GB. This suggests that reusing the previous model can sometimes outperform continuous retraining. In summary, these results demonstrate the superior performance of our method.

Memory Overhead and Scope. We further measure the additional memory footprint introduced by *LBSC*⁺. In our experiments, the training data requires about 68.5 MB, while the stored models require only 52–57 MB. For the inference reuse mechanism, the red-black tree stores 64 candidate items, requiring only 2,560 bytes. These results show that the training-related memory overhead of *LBSC*⁺ is limited. Additionally, we note that *LBSC*⁺ specifically targets cost-aware caching in storage-disaggregated cloud databases. Therefore, production factors such as query concurrency, transaction processing, and multi-tenant interference involve additional execution and resource-management mechanisms and are beyond the scope of this work.

VII. RELATED WORK

In this section, we review the relevant research works in two broad lines: caching algorithms and learning-based systems.

Caching Algorithms. The design of the cache has been studied for more than 50 years. We only focus on software cache designs here, especially with variable-sized cached data. They can be divided into two major lines of work: heuristic-based and learning-based. GDSF [29] is the heuristic-based algorithm that is the most relevant to our work. Furthermore, there are other recent researches about learning-

based algorithms such as Hawkeye [45], RL-Belady [15] and PARROT [12]. More recent learning-based caching systems exploit richer runtime features to guide admission, prefetching, and eviction under various workloads [46], [47]. For modern cloud databases, such as Snowflake [2], Alluxio [48], and PolarDB [1], they typically adopt heuristic caching algorithms (e.g., LRU and LFU), and do not consider data-fetching cost.

Learning Systems. Many system optimizations can be enabled by machine learning and deep learning models [49]–[51]. For instance, in the area of databases, such models can be applied in index optimization [52], [53], cardinality estimation [54], [55] and query optimization [56], [57]. Besides databases, ML-based works have been done in other areas of systems, such as sorting algorithms [58], bloom filter [59], and LSM-tree optimizations [60], [61]. Specifically, RusKey [60] is an reinforcement learning enhanced key-value store that supports dynamic workloads in an online manner, and they also propose a policy propagation method across levels, reducing the demand for training samples. Despite these advances toward self-driving systems, existing works do not explicitly address cost-aware caching with joint admission and eviction in cloud databases.

VIII. CONCLUSION

We proposed $LBSC^+$, a learning-based cost-aware caching framework for cloud databases that jointly optimizes admission and eviction under heterogeneous data-fetching costs. At its core, BeladySizeCost generalizes Belady’s policy by incorporating cost, size, and near-future reuse, providing principled supervision for a lightweight unified model. To ensure practical efficiency, $LBSC^+$ integrates adaptive inference reuse, cost-aware sampling, and model reuse. Experiments on synthetic traces and real cloud databases, i.e., FPDB and OceanBase, show that $LBSC^+$ reduces total data-fetching cost by up to 17% while cutting computation overhead by up to 90%. In future work, we plan to explore the integration of prefetching techniques into $LBSC^+$. Leveraging the [next-reuse](#) predictions, such an extension may further reduce data-fetching cost and latency.

REFERENCES

- [1] W. Cao, Y. Liu, Z. Cheng, N. Zheng, W. Li, W. Wu, L. Ouyang, P. Wang, Y. Wang, R. Kuan *et al.*, “Polardb meets computational storage: Efficiently support analytical workloads in cloud-native relational database,” in *FAST*, 2020, pp. 29–41.
- [2] B. Dageville, T. Cruanes, M. Zukowski, V. Antonov, A. Avanes, J. Bock, J. Claybaugh, D. Engovatov, M. Hentschel, J. Huang *et al.*, “The snowflake elastic data warehouse,” in *SIGMOD*, 2016, pp. 215–226.
- [3] X. Yu, M. Youill, M. Woicik, A. Ghanem, M. Serafini, A. Aboulmaga, and M. Stonebraker, “Pushdowndb: Accelerating a dbms using s3 computation,” in *ICDE*, 2020, pp. 1802–1805.
- [4] J. Wang, T. Li, A. Wang, X. Liu, L. Chen, J. Chen, J. Liu, J. Wu, F. Li, and Y. Gao, “Real-time workload pattern analysis for large-scale cloud databases,” *PVLDB*, vol. 16, no. 12, pp. 3689–3701, 2023.
- [5] L. V. Rodriguez, F. Yusuf, S. Lyons, E. Paz, R. Rangaswami, J. Liu, M. Zhao, and G. Narasimhan, “Learning cache replacement with cacheus,” in *FAST*, 2021, pp. 341–354.
- [6] E. J. O’neil, P. E. O’neil, and G. Weikum, “The lru-k page replacement algorithm for database disk buffering,” *ACM SIGMOD Record*, vol. 22, no. 2, pp. 297–306, 1993.
- [7] Y. Yang, M. Youill, M. Woicik, Y. Liu, X. Yu, M. Serafini, A. Aboulmaga, and M. Stonebraker, “Flexpushdowndb: Hybrid pushdown and caching in a cloud dbms,” *PVLDB*, vol. 14, no. 11, 2021.
- [8] D. Durner, B. Chandramouli, and Y. Li, “Crystal: a unified cache storage system for analytical databases,” *PVLDB*, vol. 14, no. 11, pp. 2432–2444, 2021.
- [9] “Amazon s3 select,” 2018, <https://aws.amazon.com/blogs/aws/s3-glaciarselect/>.
- [10] M. Ma, Z. Yin, S. Zhang, S. Wang, C. Zheng, X. Jiang, H. Hu, C. Luo, Y. Li, N. Qiu *et al.*, “Diagnosing root causes of intermittent slow queries in cloud databases,” *PVLDB*, vol. 13, no. 8, pp. 1176–1189, 2020.
- [11] Z. Ji, Z. Xie, Y. Wu, and M. Zhang, “Lbcs: A cost-aware caching framework for cloud databases,” in *ICDE*, 2024, pp. 4911–4924.
- [12] E. Liu, M. Hashemi, K. Swersky, P. Ranganathan, and J. Ahn, “An imitation learning approach for cache replacement,” in *ICML*, 2020, pp. 6237–6247.
- [13] Z. Song, D. S. Berger, K. Li, A. Shaikh, W. Lloyd, S. Ghorbani, C. Kim, A. Akella, A. Krishnamurthy, E. Witchel *et al.*, “Learning relaxed belady for content distribution network caching,” in *NSDI*, 2020, pp. 529–544.
- [14] L. A. Belady, “A study of replacement algorithms for a virtual-storage computer,” *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.
- [15] G. Yan and J. Li, “RL-belady: A unified learning framework for content caching,” in *ACM MM*, 2020, pp. 1009–1017.
- [16] G. Yan, J. Li, and D. Towsley, “Learning from optimal caching for content delivery,” in *CoNEXT*, 2021, pp. 344–358.
- [17] D. S. Berger, N. Beckmann, and M. Harchol-Balter, “Practical bounds on optimal caching with variable object sizes,” *POMACS*, vol. 2, no. 2, pp. 1–38, 2018.
- [18] A. MySQL, “Mysql,” 2001, <https://www.mysql.com/>.
- [19] B. Momjian, *PostgreSQL: introduction and concepts*. Addison-Wesley New York, 2001, vol. 192.
- [20] N. Beckmann, H. Chen, and A. Cidon, “Lhd: Improving cache hit rate by maximizing hit density,” in *NSDI*, 2018, pp. 389–403.
- [21] P. Brucker, “Np-complete operations research problems and approximation algorithms,” *Zeitschrift für Operations-Research*, vol. 23, no. 3, pp. 73–94, 1979.
- [22] D. Pisinger and P. Toth, “Knapsack problems,” in *Handbook of combinatorial optimization: volume1–3*. Springer, 1998, pp. 299–428.
- [23] H.-V. Nguyen and J. Vreeken, “Non-parametric jensen-shannon divergence,” in *ECML PKDD*, 2015, pp. 173–189.
- [24] J. Zhang, Z. Luo, Q. Xu, and M. Zhang, “Pa-feat: Fast feature selection for structured data via progress-aware multi-task deep reinforcement learning,” in *ICDE*, 2023, pp. 394–407.
- [25] T. Zhang, J. Tan, X. Cai, J. Wang, F. Li, and J. Sun, “Sa-lsm: optimize data layout for lsm-tree based storage using survival analysis,” *PVLDB*, vol. 15, no. 10, pp. 2161–2174, 2022.
- [26] A. Natekin and A. Knoll, “Gradient boosting machines, a tutorial,” *Frontiers in Neuroinformatics*, vol. 7, p. 21, 2013.
- [27] D. S. Berger, “Towards lightweight and robust machine learning for cdn caching,” in *HotNet*, 2018, pp. 134–140.
- [28] P. Black, “Red black tree,” 2006.
- [29] P. Cao and S. Irani, “Cost-aware www proxy caching algorithms,” in *USITS*, vol. 12, no. 97, 1997, pp. 193–206.
- [30] C. Li and A. L. Cox, “Gd-wheel: a cost-aware replacement policy for key-value stores,” in *EuroSys*, 2015, pp. 1–15.

- [31] M. Menéndez, J. Pardo, L. Pardo, and M. Pardo, “The jensen-shannon divergence,” *Journal of the Franklin Institute*, vol. 334, no. 2, pp. 307–318, 1997.
- [32] “Technical report,” https://github.com/jzx-bitdb/LBSCplus/blob/main/technical_report.pdf, 2026.
- [33] “Minio,” 2016, <https://min.io/>.
- [34] Z. Yang, C. Yang, F. Han, M. Zhuang, B. Yang, Z. Yang, X. Cheng, Y. Zhao, W. Shi, H. Xi *et al.*, “Oceanbase: a 707 million tpmc distributed relational database system,” *PVLDB*, vol. 15, no. 12, pp. 3385–3397, 2022.
- [35] D. Huang, Q. Liu, Q. Cui, Z. Fang, X. Ma, F. Xu, L. Shen, L. Tang, Y. Zhou, M. Huang *et al.*, “Tidb: a raft-based htap database,” *PVLDB*, vol. 13, no. 12, pp. 3072–3084, 2020.
- [36] W. Cao, Y. Zhang, X. Yang, F. Li, S. Wang, Q. Hu, X. Cheng, Z. Chen, Z. Liu, J. Fang *et al.*, “Polardb serverless: A cloud native database for disaggregated data centers,” in *SIGMOD*, 2021, pp. 2477–2489.
- [37] <https://github.com/jzx-bitdb/LBSCplus>, 2025.
- [38] F. Li, “Cloud-native database systems at alibaba: Opportunities and challenges,” *PVLDB*, vol. 12, no. 12, pp. 2263–2272, 2019.
- [39] P. O’Neil, E. O’Neil, X. Chen, and S. Revilak, “The star schema benchmark and augmented fact table indexing,” in *TPCTC*, 2009, pp. 237–252.
- [40] “Tpc-h,” 1988, <https://www.tpc.org/tpch/>.
- [41] J. Gray, P. Sundaresan, S. Englert, K. Baclawski, and P. J. Weinberger, “Quickly generating billion-record synthetic databases,” in *SIGMOD*, 1994, pp. 243–252.
- [42] M. Arlitt, L. Cherkasova, J. Dilley, R. Friedrich, and T. Jin, “Evaluating content management techniques for web proxy caches,” *ACM SIGMETRICS Performance Evaluation Review*, vol. 27, no. 4, pp. 3–11, 2000.
- [43] E. G. Coffman and P. J. Denning, *Operating systems theory*. prentice-Hall Englewood Cliffs, NJ, 1973, vol. 973.
- [44] N. Littlestone and M. K. Warmuth, “The weighted majority algorithm,” *Information and Computation*, vol. 108, no. 2, pp. 212–261, 1994.
- [45] A. Jain and C. Lin, “Back to the future: Leveraging belady’s algorithm for improved cache replacement,” in *ISCA*, 2016, pp. 78–89.
- [46] T. Kong, H. Li, Y. Zhao, L. Li, X. Gao, Q. Wu, and J. Cui, “Stscache: An efficient semantic caching scheme for time-series data workloads based on hybrid storage,” *PVLDB*, vol. 18, no. 9, pp. 2964–2977, 2025.
- [47] F. Zirak, F. Choudhury, and R. Borovica-Gajic, “Selep: Learning based semantic prefetching for exploratory database workloads,” *PVLDB*, vol. 17, no. 8, pp. 2064–2076, 2024.
- [48] “Alluxio,” 2021, <https://www.alluxio.io/>.
- [49] R. Zhu, L. Weng, W. Wei, D. Wu, J. Peng, Y. Wang, B. Ding, D. Lian, B. Zheng, and J. Zhou, “Pilotscope: Steering databases with machine learning drivers,” *PVLDB*, vol. 17, no. 5, pp. 980–993, 2024.
- [50] W. Wang, G. Chen, A. T. T. Dinh, J. Gao, B. C. Ooi, K.-L. Tan, and S. Wang, “Singa: Putting deep learning in the hands of multimedia users,” in *ACM MM*, 2015, pp. 25–34.
- [51] B. C. Ooi, K.-L. Tan, S. Wang, W. Wang, Q. Cai, G. Chen, J. Gao, Z. Luo, A. K. Tung, Y. Wang *et al.*, “Singa: A distributed deep learning platform,” in *ACM MM*, 2015, pp. 685–688.
- [52] S. Wu, Y. Li, H. Zhu, J. Zhao, and G. Chen, “Dynamic index construction with deep reinforcement learning,” *DSE*, vol. 7, no. 2, pp. 87–101, 2022.
- [53] T. Wang, L. Liang, G. Yang, T. Heinis, and E. Yoneki, “A new paradigm in tuning learned indexes: A reinforcement learning enhanced approach,” *SIGMOD*, vol. 3, no. 3, pp. 1–26, 2025.
- [54] K. Kim, J. Jung, I. Seo, W.-S. Han, K. Choi, and J. Chong, “Learned cardinality estimation: An in-depth study,” in *SIGMOD*, 2022, pp. 1214–1227.
- [55] J. Sun, J. Zhang, Z. Sun, G. Li, and N. Tang, “Learned cardinality estimation: A design space exploration and a comparative evaluation,” *PVLDB*, vol. 15, no. 1, pp. 85–97, 2021.
- [56] Q. Cai, C. Cui, Y. Xiong, W. Wang, Z. Xie, and M. Zhang, “A survey on deep reinforcement learning for data processing and analytics,” *TKDE*, 2022.
- [57] G. X. Yu, Z. Wu, F. Kossmann, T. Li, M. Markakis, A. Ngom, S. Madden, and T. Kraska, “Blueprinting the cloud: Unifying and automatically optimizing cloud data infrastructures with brad,” *PVLDB*, vol. 17, no. 11, pp. 3629–3643, 2024.
- [58] A. Kristo, K. Vaidya, U. Çetintemel, S. Misra, and T. Kraska, “The case for a learned sorting algorithm,” in *SIGMOD*, 2020, pp. 1001–1016.
- [59] H. Chen, Z. Wang, Y. Li, R. Yang, Y. Zhao, R. Zhou, and K. Zheng, “Deep learning-based bloom filter for efficient multi-key membership testing,” *DSE*, vol. 8, no. 3, pp. 234–246, 2023.

- [60] D. Mo, F. Chen, S. Luo, and C. Shan, “Learning to optimize lsm-trees: Towards a reinforcement learning based key-value store for dynamic workloads,” *SIGMOD*, vol. 1, no. 3, pp. 1–25, 2023.
- [61] M. Li, H. Chai, S. Luo, H. Dai, R. Gu, J. Zheng, and G. Chen, “Vega: An active-tuning learned index with group-wise learning granularity,” *SIGMOD*, vol. 3, no. 1, pp. 1–26, 2025.



Zhongle Xie (Member, IEEE) received the BSc degree from Shanghai Jiao Tong University, in 2014 and the PhD degree from the National University of Singapore, in 2020. He is currently a ZJU100 researcher with School of Software Technology, Zhejiang University. His research interests include AI4DB, vector database, and AI systems. He has served as a PC member for VLDB’ 22-24, IEEE ICDE’ 23, CIKM’ 25, VLDB’ 26, EDBT’ 26, etc.



Zhaoxuan Ji received the BSc degree from Nankai University, in 2017 and the MSc degree from the University of Chinese Academy of Sciences, in 2020. He is currently pursuing the PhD degree in the School of Computer Science & Technology at Beijing Institute of Technology, under the supervision of Prof. Meihui Zhang. His research interests include AI4DB, LLMs, and data management.



Kehan Liu (Member, IEEE) is currently pursuing an MSc degree in the School of Software Technology at Zhejiang University, under the supervision of Prof. Being Chin Ooi and Dr. Zhongle Xie. Prior to joining Zhejiang University, she earned her BSc degree from China University of Petroleum (Beijing), in 2025. Her research interests focus on caching mechanisms and performance optimization for cloud databases.



Peng Lu (Member, IEEE) received his BSc degree from Peking University and the PhD degree from National University of Singapore, in 2014. He is currently with Zhejiang University as a ZJU100 researcher. His research interests include database systems, distributed systems, and AI4DB.



Quanqing Xu (Senior Member, IEEE) received the Ph.D. degree from Peking University, Beijing, China, in 2009. He is currently with OceanBase and the Research Institute of Ant Group, Hangzhou, China. He was a research scientist with the Agency for Science, Technology and Research, Singapore, and an adjunct faculty with the Singapore Institute of Technology, Singapore. His research interests mainly include database systems and distributed systems.



Meihui Zhang (Senior Member, IEEE) received the BEng in computer science from the Harbin Institute of Technology, China, in 2008, and the PhD in computer science from the National University of Singapore, Singapore, in 2013. She is currently a professor with the Beijing Institute of Technology, Beijing, China and was an assistant professor with the Singapore University of Technology and Design (SUTD), Singapore from 2014 to 2017. Her research interests include big data management and analytics, large-scale data integration, blockchain systems and AI system. She has served as a vice PC chair/associate editor for TKDE, VLDB’18, ICDE’18, VLDB’19, VLDB’20, SIGMOD’21, ICDE’22, VLDB’23, SIGMOD’23, ICDE’23, VLDB’24, and VLDB’26.